

V. Les formulaires HTML — le début de l'interactivité

LES FORMULAIRES EN UNE PHRASE

Les formulaires HTML permettent à l'utilisateur d'envoyer des données au serveur : connexion, inscription, contact, recherche, panier e-commerce — tout passe par eux.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Structure d'un formulaire**
 - 1. Squelette minimal
 - 2. Attributs de `form`
- **B. Types d'input HTML5**
 - Exemples
- **C. Validation native**
 - 1. Attributs de contrainte
 - 2. Exemple complet
- **D. Autres éléments de formulaire**
 - 1. `textarea`
 - 2. `select` et `option`
 - 3. Cases à cocher et boutons radio
 - 4. `button`
- **E. Accessibilité**
 - 1. Associer labels et inputs
 - 2. Grouper avec `fieldset` et `legend`
 - 3. ARIA pour les messages d'erreur
- **F. GET vs POST**
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Écrire** un formulaire HTML complet avec `<form>` , `<input>` , `<label>` , `<button>` .
- **Choisir** le bon `type` d'input (text, email, password, date, file, ...) selon le besoin.
- **Appliquer** la validation native HTML5 (`required` , `pattern` , `min` , `max` , `minlength`).
- **Construire** un formulaire accessible (label associés, fieldset/legend, ARIA).
- **Distinguer** les méthodes `GET` et `POST` et savoir laquelle utiliser selon le contexte.

2. En pratique — situation professionnelle

Votre client veut un formulaire de contact sur son site. Sans connaissance des validations HTML5, vous finissez par écrire 100 lignes de JavaScript pour re-vérifier ce que le navigateur fait déjà nativement. Ce chapitre vous apprend à mobiliser le navigateur pour vous plutôt que contre vous.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Validation native (<code>required</code> , <code>pattern</code> , <code>type="email"</code>) avant tout JS	Se fier à la seule validation client (sécurité = serveur)
<code>POST</code> pour login, inscription, données sensibles	<code>GET</code> pour un mot de passe (apparaît dans l'URL et les logs)
<code><label></code> associé à chaque <code><input></code> (ally RGAA)	Remplacer <code><label></code> par <code>placeholder</code> (disparaît à la saisie)

A. Structure d'un formulaire

1. Squelette minimal

```
<form action="/inscription" method="post">  
  <label for="nom">Nom :</label>  
  <input type="text" id="nom" name="nom" required>  
  
  <button type="submit">Envoyer</button>  
</form>
```

Se connecter

à votre espace DigiShop

Email

Mot de passe

☐

Se souvenir de moi

[Mot de passe oublié ?](#)

Se connecter

Pas encore de compte ? [Créer un compte](#)

- `<form>` encapsule tous les champs.
- `action` — URL de destination où les données sont envoyées.
- `method` — `GET` ou `POST` (voir section F).
- Chaque champ de saisie est relié à un **label** via `for` / `id`.

2. Attributs de `form`

Attribut	Rôle
<code>action</code>	URL de traitement côté serveur
<code>method</code>	<code>GET</code> ou <code>POST</code> (défaut : <code>GET</code>)
<code>enctype</code>	Type d'encodage (important pour l'upload de fichiers : <code>multipart/form-data</code>)
<code>novalidate</code>	Désactive la validation native du navigateur
<code>autocomplete</code>	<code>on</code> / <code>off</code> pour l'ensemble du formulaire

B. Types d'input HTML5

Le choix du bon `type` améliore à la fois l'**UX mobile** (clavier adapté) et la **validation native** sans ligne de JavaScript.

Type	Usage	Clavier mobile
<code>text</code>	Texte générique	Standard
<code>email</code>	Adresse e-mail	Avec <code>@</code>
<code>tel</code>	Numéro de téléphone	Pavé numérique
<code>url</code>	URL	Avec <code>/</code> et <code>.com</code>
<code>password</code>	Mot de passe (masqué)	Standard
<code>number</code>	Nombre	Numérique
<code>range</code>	Curseur (0 → 100 par défaut)	—
<code>date</code> / <code>time</code> / <code>datetime-local</code>	Date, heure	Picker natif
<code>color</code>	Sélecteur de couleur	Picker natif
<code>file</code>	Upload de fichier	—
<code>search</code>	Recherche	Clavier avec <i>Rechercher</i>
<code>hidden</code>	Champ caché (valeur interne)	—

Exemples

```
<input type="email" name="email" placeholder="vous@exemple.fr" required>

<input type="tel" name="phone" pattern="[0-9]{10}" placeholder="0123456789">

<input type="date" name="naissance" min="1900-01-01" max="2010-12-31">

<input type="file" name="cv" accept=".pdf,.doc,.docx">

<input type="range" name="satisfaction" min="1" max="10" step="1" value="5">
```

♦ `placeholder` n'est pas un `label`

Le `placeholder` disparaît dès que l'utilisateur commence à taper. **Ne jamais** l'utiliser pour remplacer un label — le label reste toujours visible et accessible.

C. Validation native

1. Attributs de contrainte

```
<input type="text"
  required
  minlength="3"
  maxlength="20"
  pattern="[A-Za-z]+"
  title="Lettres uniquement, 3 à 20 caractères">
```

Attribut	Rôle
<code>required</code>	Champ obligatoire (booléen)
<code>minlength</code> / <code>maxlength</code>	Longueur min/max
<code>min</code> / <code>max</code>	Valeur min/max (pour types numériques/date)
<code>step</code>	Pas (pour <code>number</code> , <code>range</code> , <code>date</code>)
<code>pattern</code>	Expression régulière de validation
<code>title</code>	Message d'aide affiché si la validation échoue

2. Exemple complet

```
<form action="/register" method="post">
  <label for="email">Email :</label>
  <input type="email" id="email" name="email"
    required
    pattern="[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$"
    title="Adresse email au format standard">

  <label for="pwd">Mot de passe :</label>
  <input type="password" id="pwd" name="pwd"
    required
    minlength="12"
    title="Au moins 12 caractères">

  <button type="submit">S'inscrire</button>
</form>
```

Créer un compte

Rejoignez DigiShop

Prénom

Camille

Email

camille@digishop.fr

Mot de passe

• • • • •

8 caractères minimum

Date de naissance

23 / 04 / 1990



☐ J'accepte les **CGU**

S'inscrire

Déjà inscrit ? [Se connecter](#)

Si l'utilisateur soumet un formulaire invalide, le navigateur bloque l'envoi et affiche un message d'erreur natif — **sans aucun JavaScript**.

⚠ La validation côté client n'est pas une protection

Ne jamais se fier à la seule validation HTML : elle améliore l'UX, mais un attaquant peut contourner le navigateur et envoyer n'importe quoi. **Toujours re-valider côté serveur.**

D. Autres éléments de formulaire

1. textarea

Zone de texte multiligne :

```
<label for="message">Message :</label>
<textarea id="message" name="message"
  rows="5" cols="40"
  required maxlength="1000"
  placeholder="Votre message..."></textarea>
```

2. select et option

Liste déroulante :

```
<label for="pays">Pays :</label>
<select id="pays" name="pays" required>
  <option value="">- Choisir -</option>
  <option value="fr" selected>France</option>
  <option value="be">Belgique</option>
  <option value="ch">Suisse</option>
</select>
```

Sélection multiple avec `multiple` (Ctrl/Cmd + clic) et groupes d'options avec `<optgroup>`.

3. Cases à cocher et boutons radio

```
<!-- Checkbox : plusieurs choix possibles -->
<label><input type="checkbox" name="langues" value="fr"> Français</label>
<label><input type="checkbox" name="langues" value="en"> Anglais</label>

<!-- Radio : un seul choix possible (même name) -->
<fieldset>
  <legend>Sexe</legend>
  <label><input type="radio" name="sexe" value="h"> Homme</label>
  <label><input type="radio" name="sexe" value="f"> Femme</label>
  <label><input type="radio" name="sexe" value="nr"> Ne se prononce pas</label>
</fieldset>
```

4. button

```
<button type="submit">Envoyer</button>    <!-- par défaut -->  
<button type="reset">Réinitialiser</button>  
<button type="button">Action JS personnalisée</button>
```

E. Accessibilité

1. Associer labels et inputs

Trois façons :

```
←!— Méthode 1 : via for/id (préférée) →  
<label for="email">Email</label>  
<input type="email" id="email" name="email">  
  
←!— Méthode 2 : englobement →  
<label>  
  Email  
  <input type="email" name="email">  
</label>  
  
←!— Méthode 3 : aria-label (pour bouton icône sans texte) →  
<button aria-label="Rechercher">🔍</button>
```

2. Grouper avec fieldset et legend

```
<form action="/register" method="post">  
  <fieldset>  
    <legend>Identité</legend>  
    <label for="nom">Nom</label>  
    <input id="nom" name="nom" required>  
    <label for="prenom">Prénom</label>  
    <input id="prenom" name="prenom" required>  
  </fieldset>  
  
  <fieldset>  
    <legend>Contact</legend>  
    <label for="email">Email</label>  
    <input type="email" id="email" name="email" required>  
    <label for="tel">Téléphone</label>  
    <input type="tel" id="tel" name="tel">  
  </fieldset>  
</form>
```

3. ARIA pour les messages d'erreur

```
<label for="email">Email</label>
<input type="email" id="email" name="email"
  required
  aria-describedby="email-help email-error"
  aria-invalid="false">
<span id="email-help">Nous ne partagerons jamais votre email.</span>
<span id="email-error" role="alert"></span>  ←!— rempli par JS si erreur →
```

F. GET vs POST

Aspect	method="GET"	method="POST"
Où vont les données ?	Dans l'URL (<code>?nom=Robin&age=30</code>)	Dans le corps de la requête
Visibilité	⚠ Visible dans l'URL, l'historique, les logs	Non visible (mais non chiffré sans HTTPS)
Taille	Limitée (~2 000 caractères selon navigateur)	Quasi illimitée
Cacheable	Oui	Non par défaut
Idempotent	Oui (pas d'effet de bord)	Non (peut créer/modifier)
Usage typique	Recherche, filtres, navigation	Inscription, login, envoi de données sensibles

Règle : `GET` pour **lire** ou **naviguer**, `POST` pour **envoyer** ou **modifier**.

✗ Jamais de mot de passe en GET

Envoyer un mot de passe en `GET` le fait apparaître dans :

- l'URL de la barre d'adresse ;
- l'historique du navigateur ;
- les logs du serveur ;
- les *referers* des requêtes suivantes.

Toujours utiliser `POST` (avec HTTPS) pour les données sensibles.

Conclusion

1. À retenir

- Un formulaire minimal : `<form action="..." method="post">` + `<label>` + `<input>` + `<button type="submit">`.
- **Toujours associer** `<label>` et `<input>` via `for` / `id` (accessibilité + UX).
- Utiliser les **types HTML5** (`email`, `tel`, `date`, `number` ...) pour la validation et le clavier mobile adapté.
- La **validation native** (`required`, `pattern`, `min` / `max`) couvre 90 % des cas sans JavaScript.
- **Toujours re-valider côté serveur** — la validation client est une aide UX, pas une sécurité.
- Grouper les champs liés avec `<fieldset>` + `<legend>`.
- `GET` pour lire, `POST` pour envoyer/modifier. **Jamais** de données sensibles en `GET`.

2. Pour aller plus loin

- **MDN — HTML Forms Guide**
- **MDN — Form validation**
- **WebAIM — Accessible forms**
- **Constraint Validation API (validation personnalisée en JS)**